Sol Cheatsheet

Sol Cheatsheet

Quick reference for ASU's Sol supercomputer — SLURM basics, the `solx` CLI and its raw-SLURM equivalents, partitions & QOS, Sol's own wrappers, and getting at a compute-node service from your laptop.

> A rendered PDF lives at `docs/cheatsheet.pdf` (build it with `scripts/build-cheatsheet.sh`). In a terminal on Sol, run `solx cheatsheet` to print this page.

Know your access first

What partitions, QOS, and group account *you* can use — the answer drives every job decision below:

```
sacctmgr -n show assoc user=$USER format=Account,Partition,QOS
#   -> e.g.  grp_yourpi || debug,htc,private,public
myfairshare                     # dampened RealFairShare (lower = back off / use a buy-in QOS); raw `sshare -U
```

Partitions — pick by wall-time, not by "is it a GPU job?"

GPUs live in `htc`, `public`, **and** `general`. The deciding question is *how long* and *how urgently*, not CPU-vs-GPU.

Partition	Wall limit	GPUs	Use it for
`htc`	**4 h**	large A100 pool + H100/L40/A30/H200	the default for anything <=4 h, **GPU included** — least contended
`public`	7 days	A100 (+ A100-MIG, A30)	runs that need >4 h, non-preemptable
`general`	14 days	A100/H100/H200/L40	privately-owned nodes (via `-q private` or your `grp_*`)
`lightwork`	1 day	a100.20gb	the `vscode` tunnel's home; light dev
`highmem`	7 days	—	up to 2 TB RAM

QOS — priority & preemption, and which partitions accept it

QOS	Wall cap	Notes
`public`	(partition's)	default, non-preemptable

QOS	Wall cap	Notes
`debug`	**15 min**	very high priority; GPUs OK; **`public/general` only — rejected on `htc`**; one job at a time
`private`	(partition's)	preemptible access to buy-in nodes — owners can cancel you; runs past htc's 4 h
`grp_*`	up to 30 days	your group's owned nodes (if you're in one)
`class`	1 day	course users; GPU-minute caps

Routing in one line: <=4 h (incl. GPU) -> `htc` · <=15 min & urgent -> `-p public -q debug` · >4 h -> `-p public` · >4 h preemptible -> `-p general -q private`. Never `-p htc -q debug` (invalid).

SLURM basics

```
sbatch job.sh                  # submit a batch script
squeue --me                    # your jobs        (alias: myjobs; bare `sq` = whole cluster)
scancel <jobid>                # cancel
scontrol show job <jobid>      # full detail / why pending
sbatch --test-only job.sh      # validate partition/QOS/time/gres WITHOUT submitting
interactive                    # quick shell; defaults to -p htc -q public -c 1 -t 0-4
```

`#SBATCH` header skeleton (time format is `D-HH:MM:SS`):

```
#!/bin/bash
#SBATCH -p htc                 # partition  (htc = <=4h, has GPUs)
#SBATCH -q public              # QOS
#SBATCH -t 0-04:00:00          # wall-time
#SBATCH -c 8                   # cores
#SBATCH --gres=gpu:a100:1      # GPU(s)
#SBATCH --mem=64G
#SBATCH -o slurm.%j.out
```

Start from Sol's templates, don't hand-roll headers: `/packages/public/sol-sbatch-templates/templates/`.

Job stuck PENDING? Diagnose, don't spray

```
squeue --me -t PD -O "JobID,Reason:50,StartTime"   # full reason + ETA (widen Reason; don't grep '[^ ]+')
scontrol show job <id>                             # all fields for one job
```

`Reason`	Move
`Priority` (low fairshare)	priority-bound — report the ETA, **don't** resubmit
`ReqNodeNotAvail`	node unavailable (drained/down or reserved) — check the node; reroute to healthy nodes
`Resources`	capacity-bound — *now* a reroute / right-size can help

A reroute beats only `Resources`. For `Priority` / reservations every partition converges on the same start — diagnose + report, then stop submitting (resubmitting just spends more fairshare). When a reroute *is* right, modify in place: `scontrol update job <id> Partition=… QOS=…` (not `scancel` + `sbatch`, which resets accrued priority).

`solx` <-> raw SLURM

`solx` owns the interactive-allocation lifecycle; raw SLURM is the equivalent fallback for one-off reads.

`solx`	raw SLURM equivalent
`solx job start [TEMPLATE]`	`salloc` / `interactive` from a config template, *waits for the grant*
`solx job jump`	`srun --pty $SHELL` onto the compute node
`solx job list`	`squeue --me`
`solx job time`	`squeue -h -j "$SLURM_JOB_ID" -o %L`
`solx job stop`	`scancel <jobid>`
`solx keep`	renew the mtime on `/scratch` files Sol flagged (filtered by `[keep]`)
`solx job start gpu -- …`	anything after `--` is appended to `salloc` (last flag wins)

Config lives at `~/.config/solx/config.toml` (`solx config edit`). Add `--json` for machine output; `-n` to preview; `-y` to skip prompts.

Sol's own `my*` / `show*` wrappers

The `show*` wrappers are color-coded for **human eyes** and fight `awk`/`grep` — use them to *show a user*. As an **agent**, parse the right-hand form instead.

You want	Human wrapper	Parse this (agent)
Your fairshare / priority	`myfairshare`	`myfairshare` -> `RealFairShare` col
Your `/scratch` quota	—	`beegfs-ctl --getquota --uid $USER`
Your jobs right now	`myjobs`	`squeue --me -O JobID,State,Reason`
Estimated start of a pending job	`thisjob <id>`	`scontrol show job <id>` -> `StartTime=`
Efficiency of a finished job	—	`seff <jobid>`
Free capacity / partitions	`showparts`	`sinfo -h -o "%P %a %l %D %t"`
Free GPUs (= Gres − GresUsed)	`showgpus`	`sinfo -h -O "Partition,StateLong,Gres,GresUsed"`

Reaching a compute-node service from your laptop

```
# VS Code: from a Sol login node, register a tunnel (wraps srun on lightwork)
vscode                          # then open the tunnel named sol_$USER

# Manual port-forward (e.g. Jupyter on $NODE:8888), run from your LAPTOP:
ssh -N -L 8888:localhost:8888 -J $USER@login.sol.rc.asu.edu $USER@$NODE
```

`$NODE` is the compute node your allocation landed on (`squeue --me` -> NODELIST). Bind services to `localhost`, never `0.0.0.0`, on shared nodes.

Storage & caches

Path	For	Lifetime
`/scratch/$USER`	datasets, model caches, run outputs	**purged after inactivity** — renew with `solx keep`
`/home/$USER`	code, configs, `~/.local` installs	persistent, small quota

Point heavyweight caches at `/scratch`, not `/home`:

```
export HF_HOME=/scratch/$USER/.cache/huggingface
export UV_CACHE_DIR=/scratch/$USER/.cache/uv
```

Heavy I/O — where to run it

Login nodes are throttled. For a big metadata pass (e.g. touching hundreds of thousands of files) use the **DTN** (`ssh soldtn`), a **compute node** (`interactive`), or a short **`htc` batch job** — never the login node.